

# Secure UPI Payment & Conditional Settlement Platform

Customer pays over UPI. Funds are held, not released, and reach the merchant only after a verified, OTP-confirmed delivery. Return and refund handled in the same flow.

₹1,20,000 + GST

2–3 weeks

2 developers

End-to-end live demo

This is not a payment app. Payments are already solved by GPay, Razorpay and PhonePe. What we build is the **transaction-orchestration layer**: conditional settlement based on delivery confirmation, the part banks, aggregators and marketplaces actually value.

# Payments are solved. Trust is not.

The money rail already exists. The gap is settlement *conditioned* on delivery: the customer's money sits with a holding partner until the product is confirmed received, then releases to the merchant automatically, or refunds on a return.

Our platform is the **authorised API caller** that orchestrates that: it creates the payment request through the gateway, tracks the order and delivery, and instructs release or refund to the holding partner.

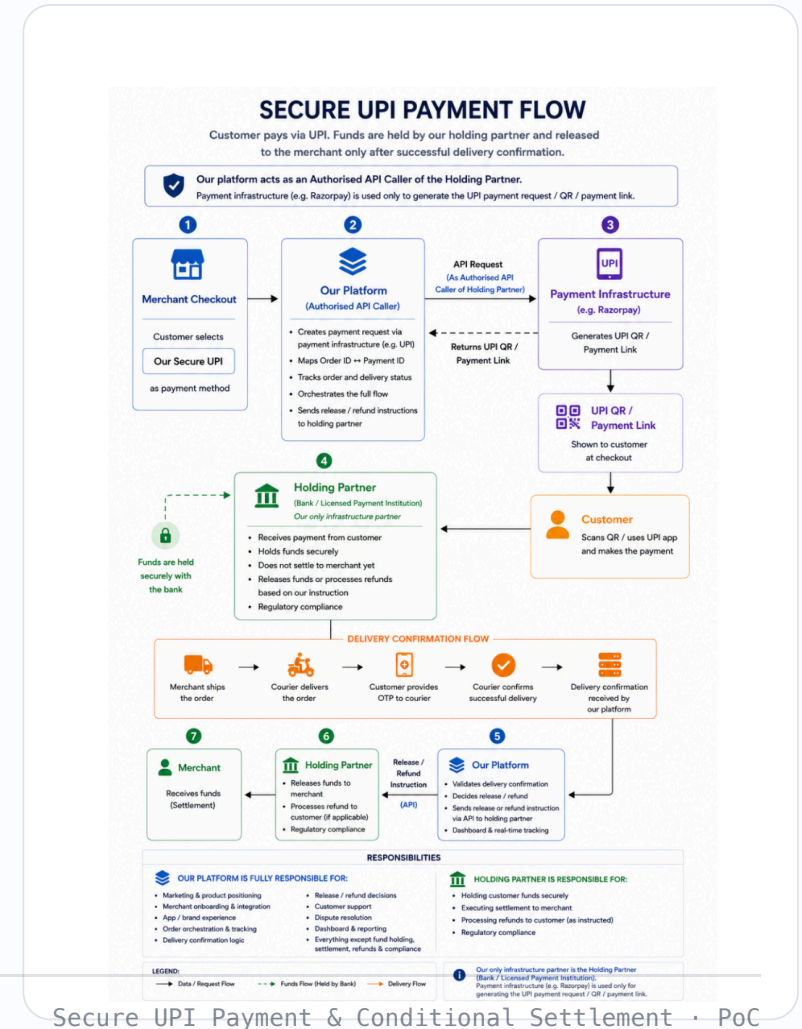
UPI collection: GPay / Razorpay

Funds held by holding partner

Release on OTP-verified delivery

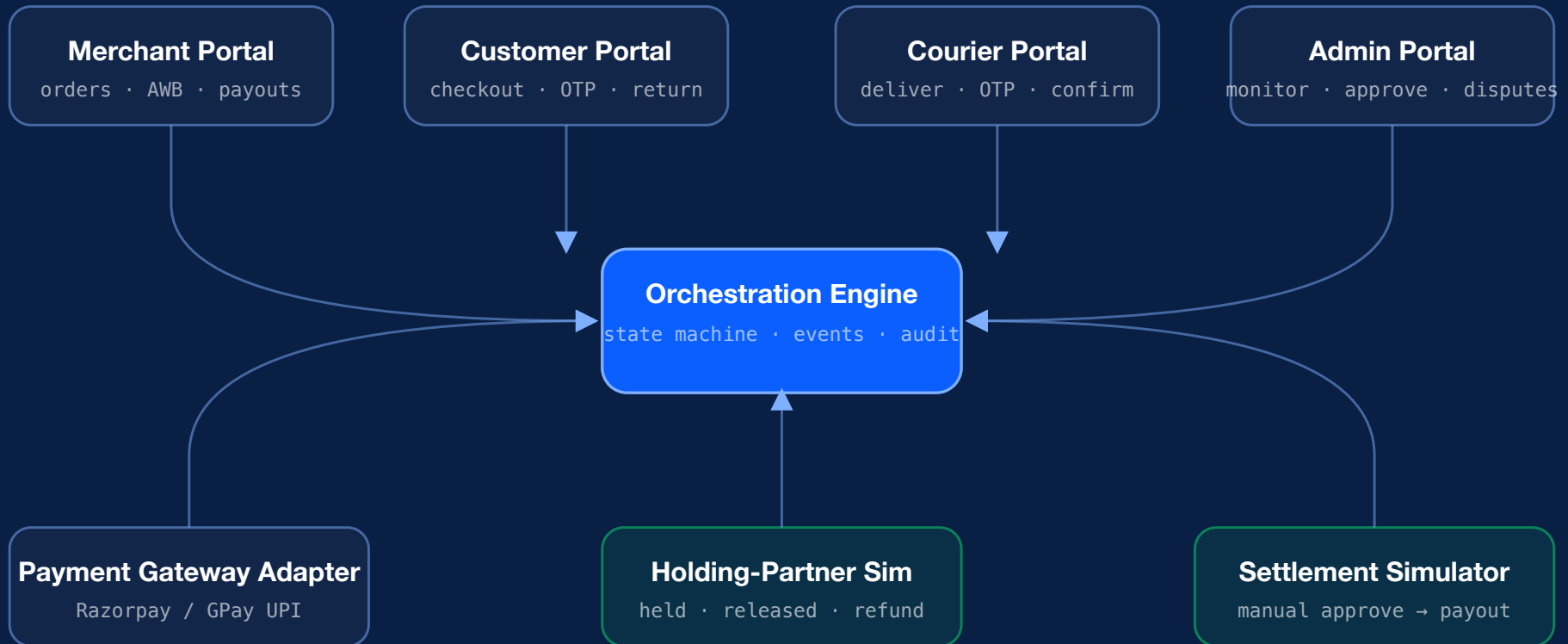
Refund on return

WE\_AINA



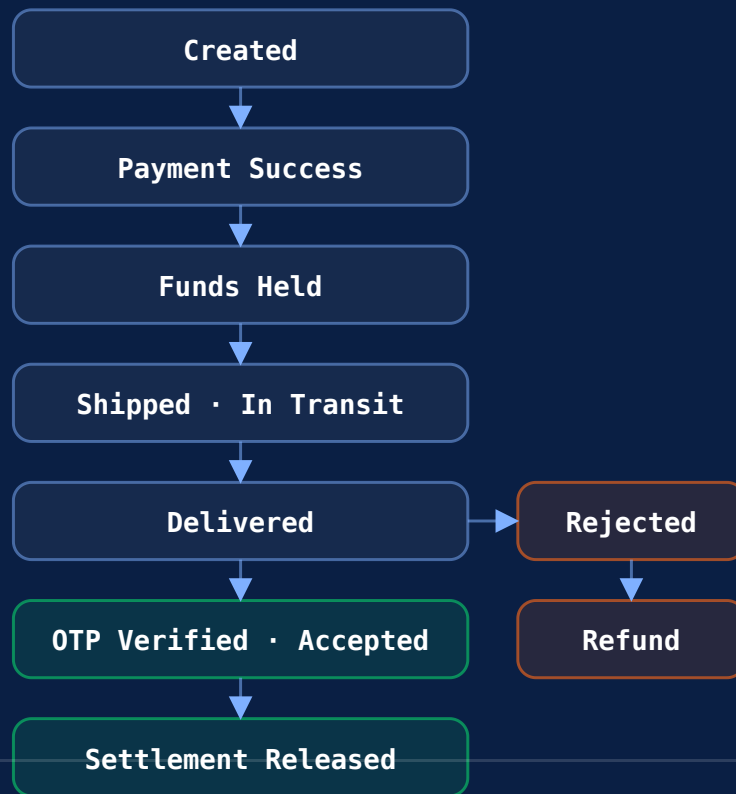
# Seven modules, one brain.

Every action passes through the orchestration engine. The four portals are what people see; the three services are what make the settlement conditional.



# The lifecycle is the product.

Every transition emits an event shown on a live timeline during the demo, so the audience watches each component react. It reads as an orchestration layer, not a status field in a database.



## EVENT STREAM

PaymentReceived FundsHeld ShipmentCreated  
Delivered OTPVerified SettlementReleased  
RefundProcessed

## API SURFACE

Roughly thirty endpoints across seven groups, one wired to a live gateway and the rest simulated. The full reference is in Appendix A.

Merchant Payment Logistics Delivery Settlement  
Customer Admin

# Four portals, three services, **one demo.**

## 1 · Merchant Portal

- Login & products
- Create order
- Secure payment QR
- Enter AWB, ship
- Settlement status

## 2 · Customer Portal

- Checkout & pay
- Scan UPI QR
- Track order
- Accept via OTP
- Return & refund

## 3 · Courier Portal

- Assigned deliveries
- Update status
- Trigger OTP
- Confirm delivered

## 4 · Admin Portal

- Transactions
- Settlement queue
- Refund queue
- Manual approval
- Audit logs

01 Merchant creates product & order

02 Platform generates UPI QR

03 Customer pays on GPay → funds held

04 Merchant ships with AWB

05 Courier marks delivered

06 Customer accepts with OTP

07 Platform validates delivery

08 Admin approves → settlement

React + Vite  
WE\_AINA

Tailwind

Node · Express

PostgreSQL

JWT

Socket.IO events

Razorpay / GPay UPI test

Google Cloud Run  
Scope & deliverables

# Fixed price. Two milestones.

| LINE ITEM                         | AMOUNT          |
|-----------------------------------|-----------------|
| UI development (4 portals)        | ₹30,000         |
| Backend APIs & orchestration      | ₹40,000         |
| Google Pay (UPI) integration      | ₹15,000         |
| Logistics & settlement simulation | ₹20,000         |
| Testing, deployment & docs        | ₹15,000         |
| Total                             | ₹1,20,000 + GST |

| MILESTONE                               | AMOUNT        |
|-----------------------------------------|---------------|
| 1 · Kick-off — design, setup, dev start | ₹60,000 · 50% |
| 2 · Final demo, deploy & handover       | ₹60,000 · 50% |

~~WE\_AINA · you talk to Chitransh or Pratik~~

| WEEK | DELIVERABLE                                                  |
|------|--------------------------------------------------------------|
| 1    | Merchant & Customer portals, order & QR generation           |
| 2    | GPay integration, holding ledger, courier portal, OTP, admin |
| 3    | Refund flow, event timeline, end-to-end test, deploy, demo   |

**Scope, stated plainly.** This is a functional Proof of Concept, not production infrastructure. Settlement is **demonstrated manually** by an admin; it does not integrate a licensed holding institution or automated settlement APIs. UPI collection uses a client-provided GPay/UPI account in test mode. Aggregator licensing, bank holding partners and automated settlement are a subsequent production phase.

₹1,20,000 + GST · 2–3 weeks · 2 developers

# The full surface.

Roughly thirty endpoints across seven groups. One initially wired to Razorpay/GPay test mode; the rest simulated.

## MERCHANT

**POST** /merchant/login  
**POST** /merchant/product  
**GET** /merchant/products  
**POST** /merchant/order  
**GET** /merchant/orders

## PAYMENT

**POST** /payment/create  
**POST** /payment/webhook  
**GET** /payment/:id  
**GET** /payment/status

## LOGISTICS

**POST** /shipment/create  
**POST** /shipment/update  
**GET** /shipment/:awb  
**POST** /shipment/delivered

## DELIVERY

**POST** /delivery/generate-otp  
**POST** /delivery/verify-otp  
**POST** /delivery/accept  
**POST** /delivery/reject

## SETTLEMENT

**POST** /settlement/hold  
**POST** /settlement/release  
**POST** /settlement/refund  
**GET** /settlement/status

## CUSTOMER

**POST** /customer/login  
**GET** /customer/orders  
**POST** /customer/return  
**POST** /customer/dispute

## ADMIN

**GET** /transactions  
**GET** /events  
**GET** /audit  
**GET** /dashboard

# Five core tables.

Every row is an auditable fact. The event table is the source of truth the live timeline reads from.

## orders

- id
- merchant
- customer
- amount
- status
- awb
- payment\_id
- otp

## payments

- payment\_id
- gateway
- amount
- status
- held
- released
- refunded

## settlement

- settlement\_id
- payment\_id
- status
- release\_time
- approved\_by

## logistics

- awb
- carrier
- status
- otp
- delivery\_time

## events

- type
- ref\_id
- payload
- ts
- actor

Stack: PostgreSQL (SQLite for the fastest MVP), Prisma ORM, Node/Express, Socket.IO for the live event stream. Optional Redis + BullMQ if the settlement queue is made asynchronous.

WE\_AINA



# Return & refund, same rail.

01 Customer presses **Return** and picks a reason

02 Settlement stays **Held**, never released

03 Merchant approves the return

04 Platform calls /settlement/refund

05 Holding partner refunds the customer

06 Timeline shows **Refund Completed**

WE\_AINA

## WHY THIS WINS THE ROOM

The payment is not the pitch, the **conditional hold** is. This is exactly what a bank, a payment aggregator or a marketplace escrow product needs to see working before they fund the licensed version.

The demo proves the orchestration layer end to end. The production phase swaps the simulator for a licensed holding partner and automated settlement APIs, and the same state machine carries real money.

# Approve, and we **kick off this week.**

Milestone 1 starts the build. In two to three weeks you have a working, end-to-end conditional-settlement demo you can put in front of a bank or an aggregator, and decide the production phase from something real instead of a slide.

₹60,000 to start

Live demo in 2–3 weeks

Source, DB & deploy handed over

We do not sell you a team and a steering committee. It is two accountable partners and an agent fleet, a fixed number, and a working system. That is the whole offer.